

HarborFlow Assignment 2

Team delivery and study protocol

DIT014 • Port Intelligence Console • Prepared 6 October 2026

A ten-workday route from starter files to one tested product. Four students, 40 hours each. Every step also says what to learn and what evidence proves it works.

1. Agree on the plan before writing code

Proposed window: Tuesday 6 October to Monday 19 October 2026. These are ten weekdays across two weeks. The assignment PDF does not specify the official submission deadline: check Canvas on Day 1 and move this window if needed. This is a planning target, not a claim about the official deadline.

Budget assumption: each person works 20 hours per week, four hours per weekday, giving 40 hours per person and 160 person-hours for the team. Course study related to this assignment is included. The team has four people working alongside each other; that does not create a 160-hour individual schedule.

The supplied uploads contain the Python starter modules and CSV files, but no vessels/, manifests/, shipments/ or incidents/ JSON folders. Get the complete starter ZIP from Canvas before live-data implementation. Preserve its folder structure and intentional errors.

- Replace Member B/C/D with real names. Create the CodeGrade group with exactly these four members.
- Confirm the official deadline, submission rules and latest TimeEdit schedule. The saved syllabus says TimeEdit is authoritative and announcements may change sessions.
- Do not add optional features: every Task 0–9 is compulsory for G. The assignment is G/U, with no grade tiers.
- Do not start a tutoring session until Ray writes LOGIN. This document is the planning protocol, not a completed teaching session.

The product in one picture, explained in words

Think of each vessel record as an identity card. A port call is a booked visit. Its manifest is the cargo list. Shipment records describe each cargo item. Incidents describe things that happened. Weather describes conditions at a port on a date. The identifiers are the labels that let the program connect those cards correctly.

Connection	What the program follows
Vessel → call	vessel_id
Call → manifest	call_id
Manifest → cargo	shipment_ids → shipment_id, keeping list order
Call → incident	call_id
Call → forecast	port_code AND arrival_date/date

2. Divide effort equally, not just task numbers

Each owner is responsible for a small part of the shared product. Ownership means building it, proving it works and teaching someone else how it works. Nobody owns the whole README or all testing alone.

Member	Primary build work	Second-person review
--------	--------------------	----------------------

Ray (A)	Task 0 menu; Task 1 vessels; Task 5 month report; Task 7 stable sequence. Shared menu/date helpers.	Reviews B: manifest links and priority rules.
Member B	Tasks 2 and 3; manifest/shipment loaders and ID resolution.	Reviews C: customer updates and weather joins.
Member C	Tasks 4 and 8; CSV reading/writing and weather conversions.	Reviews D: regex and index/cache behavior.
Member D	Tasks 6 and 9; incident loader, sanitizer and index/cache.	Reviews Ray: loop, vessel validation, month sorting, sequence.

Task 9 is a larger feature, so D gets fewer tasks. Ray gets more smaller tasks. B and C also build shared loaders. The equal target is time and responsibility, not the count of menu options.

Use of time	Hours per member	Team person-hours
Study: concepts, course sessions or lab practice	10	40
Build assigned features and shared helpers	18	72
Review and integration	5	20
Acceptance tests, README and handover	5	20
Coordination and decisions	2	8
Total	40	160

These are planning estimates. Track actual hours each day. If one person has much more work, move a small function or test package to someone with spare build hours and record the changed owner. Keep each person at 40 hours; do not solve delays by quietly giving Ray everyone's work.

3. Keep the team moving

- Write a working agreement: common work window, primary chat channel, response expectations, owners, reviewers, daily integration time and what happens when a commitment is missed.
- Use one board: Ready → Building → Review → Integrated → Verified. Keep only one feature in Building per person.
- At the start, each person gives one sentence: yesterday's result, today's goal, current blocker. Keep the daily check-in to about 12 minutes within the four-hour budget.
- Use one Git branch per small feature. Commit a runnable change, request review, then merge. Pull the current shared version before starting the next change. The reviewer runs the feature as well as reading it.
- Never replace an entire shared module with an LLM-generated version. Own specific functions. Coordinate changes to data_access.py and operations.py before editing the same area.
- The owner adds test-ledger.csv rows; the reviewer checks them. Coordinate CSV edits so one merge does not overwrite another person's evidence.
- A feature is done only after its reviewer approves, it runs through main.py, evidence is recorded, and both people can explain it.
- After 30 minutes blocked, write what you expected, what happened and what you tried. Ask the team. If a delivery risk remains unresolved, contact teaching staff early; record the risk and proposed next step.
- Disagree using a test or a source requirement. For an unclear contract, record the ambiguity, propose alternatives and ask the teacher. Do not invent an exact required output.
- If requirements change, record the change, affected behavior and tests, smallest update, risk and trade-off. Rerun earlier tests to protect existing behavior.

One repeatable feature protocol

Step	Action	Evidence before continuing
1. Understand	Read that task. Explain the staff problem in one sentence.	A short description in your own words.
2. Define	List inputs, outputs, links, exact text and error behavior.	A requirement checklist with expected outputs.
3. Plan	Write pseudocode: the steps without Python syntax. Agree function inputs/returns.	Reviewer confirms the plan and interfaces.
4. Learn	Read relevant lecture material; do a tiny exercise outside the product.	Predict the result and explain why it occurs.
5. Build	Implement one small function at a time. Separate reading, deciding and printing.	A normal example runs without hard-coded sample IDs.
6. Verify	Try normal, boundary, missing and malformed data where applicable.	Actual results recorded in the test ledger.
7. Review and integrate	Reviewer checks failure paths; merge; run from the real menu.	Integrated feature plus regression checks.
8. Explain and improve	Trace input → file → rule → output. Explain one design decision.	A teammate can repeat the trace; README updated.

4. Two-week execution plan

Each row is one four-hour workday per person, including study and coordination. The timetable is a route, not permission to skip a failing requirement. Days 8–10 protect review and delivery time.

Day / date	Shared checkpoint and learning	Ray	B	C	D
1 • Tue 6 Oct	Get full ZIP; confirm deadline/group; working agreement; read Tasks 0–9; sketch links and function contracts. All study dictionaries/files basics.	Exact menu skeleton.	Manifest/shipment field map.	CSV fields and export plan.	Incident fields and cache plan.
2 • Wed 7 Oct	Lecture 11: dictionaries. Build/review loaders first. Gate: mixed valid/bad records can be read safely.	Task 1 loader + output.	Manifest/shipment loaders.	Calls/weather CSV loaders.	Incident loader; define index shape.
3 • Thu 8 Oct	Learn files, exceptions, modules ahead of their classes. Shared loader behavior reviewed before dependent features.	Task 0 errors and Task 5.	Task 2 selection and joins.	Task 4 first export.	Task 6 regex patterns + preservation.
4 • Fri 9 Oct	Small-feature integration. Check exact outputs. Study nested records and rule order.	Task 5 invalid dates/sorting.	Task 2 errors, missing links.	Task 4 update + reload.	Task 6 no-match + output path.
5 • Mon 12 Oct	Lab 11: dictionaries/nested data. Gate: menu and Tasks 1,2,4,5,6 run together. Start algorithms.	Task 7 hand trace + function.	Task 3 pure classification.	Task 8 join + weather rules.	Task 9 normalization/index.
6 • Tue 13 Oct	Lecture 12 + optional Open Lab. Focus on failure boundaries.	Task 7 repeat/tie cases.	Task 3 precedence/resolutions.	Task 8 thresholds/errors.	Task 9 cache save/load/staleness.
7 • Wed 14 Oct	Lab 12 and Lecture 13 occur today. Use budget rule below. Gate: all Tasks 0–9 connected to main.py.	Wire menu and sequence.	Review Ray; resolve integration.	Review D; weather completion.	Review C; search scoring/order.

8 • Thu 15 Oct	Feature freeze: only required gaps/defects. Cross-review and acceptance campaign. All explain a feature they did not write.	Review B; menu/month tests.	Relationship/priority tests.	Export/weather persistence tests.	Sanitizer/search/cache tests.
9 • Fri 16 Oct	Lab 13: modules/CSV/JSON. Clean-folder rehearsal; repeated runs; finish concrete contribution records.	Clean entry/path checks.	Missing/malformed campaign.	CSV reload/update campaign.	JSON cache/sanitized output campaign.
10 • Mon 19 Oct	Lecture 14: regex/course integration. Final regression + four individual demonstrations. Submit early enough to react to CodeGrade.	Check final menu contract.	Check group/modules.	Check README/ledger.	Check outputs/cache.

Daily four-hour rhythm

Default day: 12 minutes check-in, 60 minutes study, 108 minutes build, 30 minutes review/integration, 30 minutes tests/documentation. Total: 240 minutes. Take short pauses inside those blocks. Move minutes between blocks when a class takes longer; the total stays four hours.

A listed lecture or lab lasts 105 minutes. Attending one uses 45 extra minutes compared with the default study block: take those minutes from build time. On days with no scheduled class, use a shorter study block to recover the build time while staying within four hours. The ten-hour study allowance can cover five 105-minute sessions plus 75 minutes of targeted preparation.

Suggested attendance within that allowance: Lecture 11 (7 Oct), Lab 11 (12 Oct), Lecture 12 (13 Oct), Lab 13 (16 Oct), Lecture 14 (19 Oct). Learn the missing file/module/regex concepts early from the relevant course materials and individual lab tasks. This is a budget recommendation, not a claim about compulsory attendance. If the course requires more attendance, reduce available build time and rebalance the plan.

14 October has both Lab 12 and Lecture 13, together 210 minutes. Attending both leaves only 30 minutes that day for the project. Do not also book a full build day. Reallocate hours from other days or use the relevant materials for self-study when permitted. Optional open labs also consume the same budget. Protect other courses separately.

Progress gates and recovery

- Day 2: agreed and reviewed loader interfaces. If blocked, use tiny artificial records in a separate test folder so business-rule work can proceed, then test against the complete unmodified starter dataset.
- Day 5: a useful integrated subset. No isolated task scripts that bypass the shared menu.
- Day 7: every service is reachable. If not, assign the missing compulsory work first and cancel optional polish.
- Day 8: do not add enhancements. Transfer a small unfinished subtask to spare capacity; keep its original owner involved for understanding.

- Day 9: clean-folder rehearsal passes. Unresolved compulsory failures remain visible in the ledger and must be fixed for G.
- Day 10: all four explain the product; submit correct files under the correct group. If Canvas has an earlier deadline, move these gates earlier.

5. Study protocol linked to lectures and labs

Study a concept, prove it on a tiny example, then use it in the product. This keeps learning attached to something useful. Lecture/lab titles below come from Syllabus FOP.pdf; the detailed exercise sheets and full slide sets were not uploaded, so exact exercise-number mapping needs those materials.

Course pair / saved schedule	Concept in plain terms	Project use / proof of understanding
L04 • 9 Sep; Lab04 • 11 Sep: Conditionals and Boundaries	A condition chooses an action; a boundary is where the answer changes.	Tasks 3,8: explain what happens exactly at 5000 kg, 45 km/h, 2.5 m, 8 mm.
L05 • 15 Sep; Lab05 • 16 Sep: For Loops and Accumulation	Visit each record and build a total.	Tasks 1–5,8,9: total weight, counts and word matches.
L06 • 16 Sep; Lab06 • 18 Sep: While Loops and Validation	Keep asking until input is valid or the menu is closed.	Task 0 and retry prompts: invalid date repeats the date only.
L07 • 21 Sep; Lab07 • 22 Sep: Functions and Decomposition	Give one named function one clear job.	All tasks; classify one shipment without asking/printing/reading files.
L08 • 23 Sep; Lab08 • 25 Sep: Lists and Tuples	An ordered list keeps cargo/events in the correct sequence.	Tasks 2,7: preserve manifest order; distinguish a continuous slice from selected scattered events.
L09 • 28 Sep; Lab09 • 29 Sep: Recursion and Call Stacks	Understand how calls run and return.	Trace menu → service → loader → return. Recursion is not required by Assignment 2; do not force it into the product.
L10 • 1 Oct; Lab10 • 2 Oct: Code Quality, Testing and Responsibility	Show evidence; keep changes understandable.	Every task: expected/actual results; focused errors; review and honest contribution records.
L11 • 7 Oct 10:15–12:00 Babord; Lab11 • 12 Oct 10:15–12:00 Alfons	Dictionaries and Nested Data: find information by a key, including lists inside records.	Tasks 1–9: build ID lookups; resolve shipment IDs; make word → incident IDs index.
L12 • 13 Oct 10:15–12:00 Babord; Lab12 • 14 Oct 08:15–10:00 Alfons	Files, Paths and Exceptions: open the right file and handle expected failures.	Loaders, safe exports and cache; show one bad record does not hide valid records.
L13 • 14 Oct 10:15–12:00 Babord; Lab13 • 16 Oct 13:15–15:00 Alfons	Modules, Directories, CSV and JSON: separate jobs and read/write structured files.	Reusable modules; customer CSV; incident-index JSON; reload both on a later run.
L14 • 19 Oct 10:15–12:00 Babord; Lab14 • 23 Oct 13:15–15:00 Alfons	Regular Expressions and Course Integration: identify text shapes and combine the course skills.	Tasks 6,9: sanitize sensitive patterns; normalize words. Learn early; use L14 to check. Lab14 follows the proposed handover.

Additional saved sessions: Open Lab, 13 October 13:15–15:00, Alfons; Oral exam Part 1, 20 October 10:15–12:00, Alfons; Oral exam Part 2, 21 October 10:15–12:00, Babord. These are syllabus listings, not a statement about each student's attendance or assessment obligation.

A 60-minute study block

- 0–10: read the relevant slides or lab brief. Write the concept and one unfamiliar term.
- 10–25: trace a tiny example on paper. Predict its output before running it.
- 25–45: write the tiny example yourself; run it; explain any difference.
- 45–55: connect it to today's HarborFlow function; write pseudocode and one normal/failure test.
- 55–60: record what you can now explain and the remaining question. If time is short, continue the same concept tomorrow.

For tutor-led sessions, Ray uses LOGIN to start. Keep each session at 90 minutes or less and use the agreed teaching template. Work one concept at a time with short questions. At LOGOUT, record the demonstrated objective, exercise, difficulty, evidence and next prerequisite. An unfinished objective becomes an explicit pending item or is reduced to an achievable objective within the time.

Small practice tasks before product work

Topic	Tiny exercise	Ready when you can...
Dictionary joins	Use two made-up vessel records and three call records. Find the vessel name using vessel_id.	Explain why filenames are not reliable business identifiers.
Files and exceptions	Read one valid JSON file, one broken JSON file and one missing file in a separate sandbox.	Catch the correct exception and still process the valid record.
CSV persistence	Create two fictional customers, update the first, close and reload.	Keep exactly two rows and one header.
Regex	Try a container, seal and email separately, then all three in a report with line breaks.	Replace only matched values and preserve other text.
Sliding window	Trace A,B,A,C,B with start position and most recent positions.	Explain why the best length is 3 and the first winning sequence is B,A,C.
Inverted index	Use two fictional reports: "delay delay" and "reefer delay". Search "delay delay reefer".	Count distinct query words: scores 1 and 2; no duplicate IDs per word.

6. Function boundaries: agree before coding

An interface is an agreement: what a function receives and what it gives back. Think of a plug and socket. If both people agree on the shape, their parts can connect without rewriting each other's work.

Module	Responsibility	Suggested interface, not a required assignment name
main.py	Exact menu/prompts, retry loops and orchestration.	main(); print_menu(); prompt_date(); small service handlers
data_access.py	Paths, file reads/writes, required fields and useful rejected-record reports.	load_json(path) → record; collection loaders → (valid_records, issues); load_calls/load_weather → records and issues
operations.py	Cargo links, classification, customer totals, month selection and forecasts.	select_call(...); resolve_manifest(...); classify_shipment(record) → (category, reason); profile_customer(...); weather_recommendations(...)
text_tools.py	Text transformations, stable sequence, word index and search.	sanitize(report) → (text, changed); longest_stable(events) → sequence; normalize(text) → words; build_index(...); search(index, titles, query)

Agree issue-reporting ownership so the same error is not printed twice. Low-level readers can raise specific exceptions; collection readers catch useful file/record errors and return valid records plus issues. Business functions should receive data and return decisions. User-facing handlers print the required interface.

Do not catch every Exception. Handle FileNotFoundError, json.JSONDecodeError, csv.Error and ValueError at boundaries where you can give a useful response. A coding bug must remain discoverable. Use only Python's standard library; pandas and other third-party packages are forbidden. Avoid assumptions that depend on the supplied examples.

7. Task cards: build, then prove

Use the feature protocol in Section 3 for every card. Exact prompts, punctuation, capitalization, ordering, filenames and output formats in the assignment are the contract. Copy the PDF's templates when implementing; this document does not replace that source.

Task 0 • Product shell • Ray • Assignment p.4–5

- Build: copy the exact menu; keep a loop-state variable; dispatch options 1–9; option 10 sets the loop inactive and prints the required closing line. Do not use exit() or stop the menu with break.
- Validate: non-integer and out-of-range choices print "Error - Select a service from 1 to 10." Recoverable service errors return safely to the menu.
- Prove: text, 0, 11 and a valid option; invoke every service then return; choose 10 and confirm the exact close message.

Task 1 • List vessels • Ray • p.5

- Build: read every .json in dataset/vessels; require vessel_id, name, imo, capacity_teu; report and skip malformed or incomplete records; sort valid names case-insensitively.
- Print: heading, one vessel per line with IMO and comma-separated TEU, then skipped-file count. Do not assume the example count is always one.

- Prove: mixed valid/bad files, missing required field, case-insensitive name order and a different dataset.

Task 2 • Inspect manifest • B • p.5

- Build: display registered vessels; trim/case-insensitively match name; ask date; invalid YYYY-MM-DD repeats only date. Find the earliest call on/after it; equal dates use smallest alphabetical call_id.
- Resolve manifest by call_id and shipments by fields, not filenames. Require the manifest fields in the dictionary. Keep manifest shipment order. Count missing records separately; sum resolved weight; show declared count.
- Print: day month year using datetime; weight lines as in PDF and total with two decimals. Missing optional flags mean false; absent cargo_type prints NOT PROVIDED. Missing linked cargo prints MISSING RECORD.
- Prove: unknown vessel, invalid date, no upcoming calls, unreadable/invalid manifest, equal-date calls, renamed JSON filename, missing shipment and optional fields. Return to menu for the specified not-found/unavailable cases.

Task 3 • Priority cargo • B • p.6

- Build: trim/case-insensitively match call ID. For resolved shipments only, a pure function returns category and reason. First match wins: hazardous → CONTROLLED; temperature_controlled → COLD CHAIN; EXPRESS or weight >= 5000 → EXPEDITE; otherwise STANDARD.
- Print: manifest order; category totals in CONTROLLED, COLD CHAIN, EXPEDITE, STANDARD order. Unknown call prints Port call not found.; unreadable/invalid manifest prints Manifest unavailable.; return to menu.
- Prove: flags combined, EXPRESS below 5000, standard at 4999.99/5000, absent flags, missing shipment and unknown call. Confirm the exact reason wording against the assignment; the PDF examples do not specify every reason phrase.

Task 4 • Customer profile export • C • p.6

- Build: match trimmed customer names case-insensitively across all valid shipment files. Count shipments/EXPRESS; sum weight; sort unique destinations and join with |.
- Write exports/customer-profiles.csv with exact header once. New customer appends; existing customer replaces its row using an in-memory list and safe rewrite without duplicates or losing other rows. Weight has two decimals.
- Prove: first write, second customer, update same customer with different case/spaces, reload CSV, unique destination sorting. No match prints Customer not found. and neither creates nor modifies the file. Verify update message/path.

Task 5 • Month report • Ray • p.6–7

- Build: month integer 1–12; invalid input prints Error - Enter a month from 1 to 12. and repeats only that prompt. Read calls, validate dates, resolve vessel names; report/skip invalid dates and unknown vessel_id.
- Print: matching months across scheduled records, sorted by arrival_date then vessel name. Use month/date presentation in the PDF. No valid matches prints No port calls found.
- Prove: text/0/13, month 1/12, invalid source date, unknown vessel, equal dates and no matches.

Task 6 • Sanitize incident • D • p.7

- Build: titles sorted case-insensitively and numbered from 1. Invalid list selection prints Error - Select a listed incident. and repeats only selection.

- Use re.sub for container = four letters + seven digits; seal = SEAL- + six digits; both case-insensitive. Email uses the stated local/domain character sets and a final dot plus at least two letters. Replace with [CONTAINER], [SEAL], [EMAIL].
- Preserve all other text, punctuation and line breaks. At least one substitution means changed. Create exports/sanitized if necessary; save <incident_id>-sanitized.txt only when changed. No match prints Incident contains no share-sensitive identifiers.
- Prove: each identifier alone, mixed line, mixed case, several replacements, multiline preservation, no match and invalid selection. Avoid matching partial larger identifiers unintentionally; check any unspecified boundary interpretation with the teacher.

Task 7 • Stable event sequence • Ray • p.7–8

- Build: select a call; use its manifest event_codes. A sliding window is a continuous slice with a moving start/end. Store most recent positions in a dictionary; a repeat inside the slice moves the start just after the previous position.
- Keep the first equal-length maximum: replace the best only when the new length is larger. Implement this yourself using standard Python. Print event list, winning length and arrow-separated sequence as specified.
- Prove: all unique, immediate repeat, repeat outside active window, tied maxima; also test empty/single lists where relevant. The PDF does not prescribe all selection/error prompts here: agree consistent behavior and record it.

Task 8 • Weather risk • C • p.8

- Build: trimmed/case-insensitive vessel selection; repeat only invalid date. Upcoming means on/after date. Report/skip bad call dates. Match forecast on port AND date. Sort calls by date then call_id.
- Convert numeric fields in a focused try block; report malformed forecast rows. Missing forecast prints Forecast unavailable and processing continues. Unknown vessel prints Vessel not found.; no valid upcoming calls prints No upcoming port calls found.
- Apply ALL true recommendations, in table order: wind >=45 suspend cranes; waves >=2.5 tug assistance; rain >=8 AND wind <45 covered handling; all three below thresholds gives normal plan.
- Prove: exactly/just below each threshold; wind+waves; waves+rain with wind<45; heavy rain with wind>=45; normal values; malformed numeric row; missing forecast; equal-date call order.

Task 9 • Incident search/cache • D • p.9

- Build: normalize report/query with the same lowercase regex word rules. Index word → incident IDs; lookup incident ID → title. No duplicate ID in a word list even if a word repeats.
- Persist dataset/incident-index.json with top-level index and titles; sort word keys, title keys and ID lists. Rebuild if missing, malformed or older than any source incident file; otherwise reuse.
- Search only the loaded index/title lookup, not reread reports. Score by DISTINCT query words matched; show only positive scores; sort descending score then incident ID. Follow the PDF output template.
- Prove: single word, repeated query word, punctuation/case, score tie, no match, missing cache, corrupt cache, newer source, fresh-cache reuse and reload on another run. The no-match message is not specified in the PDF; agree and document it.

8. Exact interface and persistence checkpoints

Task 0 menu: copy exactly

HARBORFLOW PORT INTELLIGENCE

1. List registered vessels
2. Inspect a vessel manifest
3. Identify priority cargo
4. Export a customer operations profile
5. Find port calls by month
6. Sanitize an incident report
7. Analyze the longest stable event sequence
8. Assess weather risk for upcoming calls
9. Search incident reports
10. Close console

Select service:

Close message: Console closed. HarborFlow operational data remains safe.

Artifact	Contract
exports/customer-profiles.csv	customer,shipment_count,total_weight_kg,express_count,destinations Exactly one header; two weight decimals; update preserves other rows.
exports/sanitized/<incident_id>-sanitized.txt	Only create when at least one substitution occurs; preserve remaining text.
dataset/incident-index.json	Only permitted dataset write in the starter instructions. Keys index and titles; deterministic sorting; valid reloadable JSON.
Supplied dataset	Read-only otherwise. Never repair/delete intentional malformed or missing records. Test variations in a separate temporary copy.
readme.txt	Four names, concrete contributions, run command, tested Python version, architecture, paths, errors, safe updates and limitations.
test-ledger.csv	area,test_case,input_summary,expected_result,actual_result,owner,status

Keep customer names and incident text out of extra debug output. Record useful test evidence without adding unnecessary sensitive details. Output examples in the PDF are examples: do not hard-code sample vessels, customers, counts, IDs or dates.

Contract questions to resolve early

The PDF does not define every diagnostic for skipped records, every Task 3 reason, every Task 7 prompt or every Task 9 empty-result message. Preserve all specified text exactly. For remaining choices use consistent behavior, record the assumption, and seek clarification where it could affect automated tests. Do not present your own choice as an official requirement.

9. Test evidence and definition of done

An acceptance test asks whether the product meets a client requirement. A boundary test checks the exact place a rule changes. A regression test reruns an earlier check after a change. Write the expected result before running; record the actual result honestly afterwards.

Area	Minimum campaign
Files	Missing file; malformed JSON; invalid CSV number; mixed valid/invalid records.

Relationships	Known IDs; missing shipment; unknown vessel/customer; filenames different from identifiers.
Export	First write; second customer; update existing row; generated CSV reload; no-match leaves file unchanged.
Regex	Each type alone; mixed identifiers; no match; preserve punctuation/newlines.
Sequence	All unique; immediate repeat; repeat outside active window; first tie kept.
Weather	Each threshold; combined recommendations; absent forecast; malformed number; invalid call date.
Search	Single word; repeated query word; no match; score/ID ordering; corrupt cache rebuild; stale/fresh cache.
Interface	Invalid menu; local retry prompts; service return; exact text/order/filenames; option 10 closes.

Use the supplied ledger header unchanged. Add cases rather than treating its starter rows as the entire campaign. Keep owner/status/expected/actual filled. Suggested statuses: Not run, Pass, Fail, Blocked. A failure stays open until retested successfully. Every member owns evidence and performs a second-person review.

Final clean-folder rehearsal

- Make a fresh project folder with the complete original starter dataset and only the intended program modules. Start from that root with python main.py.
- Run all ten menu options. Compare specified prompts, punctuation, ordering, numeric formats and filenames character by character.
- Generate first exports/cache. Close and start again. Reload them and update a customer; preserve other customers and avoid duplicates.
- Confirm all deliberate bad/missing records produce useful recoverable behavior and later valid records still work.
- Remove debug prints, temporary tests/data and machine-specific paths. Keep real test evidence.
- Each member demonstrates a feature they did not author and traces one input through loader, rule, output and failure handling. Ask about any module; everyone must understand the integrated product.
- Confirm CodeGrade group of exactly four. Submit main.py, every imported Python module, readme.txt and test-ledger.csv. Do not submit altered datasets or sample output files unless CodeGrade explicitly requests them.
- Inspect feedback and resubmit before the deadline where allowed. Record the final group submission and remaining known limitations. Do not claim compulsory requirements pass if they still fail.

Concrete contribution entry

Use: “Built [functions/tasks]; tested [case names]; reviewed [teammate’s feature]; participated in [integration/design decision]; explained [end-to-end flow].” Generic claims such as “everyone did everything” do not show contribution. Document permitted AI assistance and verify its suggestions according to course guidance.

Short retrospective

Each person answers: Which commitment did we keep? Where did communication/integration create risk? Which decision improved reliability? What will we change next time? Record useful answers, not blame.

10. Running the same protocol with one person and an LLM

The structure works for one person: follow the same dependency order, but process one feature at a time. An LLM can help with explanations, pseudocode, review and test ideas. It does not become a team member or replace each student's contribution and understanding.

Important capacity limit: the group plan provides 160 person-hours, while one person on the same schedule has 40. Completing the full product alone in 40 hours may be possible for someone already comfortable with these concepts, but cannot be promised for a beginner. Keep all compulsory requirements; reassess the deadline or available hours if tests show the scope does not fit. Do not disguise solo/AI work as four people's work.

Solo build order

1. Inventory/contracts → 2. menu and loaders → 3. vessel list/month report → 4. manifest/priority → 5. customer export → 6. weather → 7. sanitizer → 8. stable sequence → 9. search/cache → 10. full regression and handover. Finish and verify one small function before opening the next.

Reusable agent prompt

You are my learning and implementation assistant for HarborFlow Assignment 2. Treat the assignment PDF, START-HERE.txt and DATA-DICTIONARY.md as the contract. Use only Python's standard library. We are currently on Day [number], Task [number], function [name]. I will supply the current code, relevant requirement and expected/actual result.

First ask me to explain the goal and predict one normal example. Then help me list inputs, outputs, links, exact messages and failure cases. Give a small hint or pseudocode before complete code unless I explicitly request implementation. Work on one function at a time. Preserve shared interfaces, starter structure and source data. Do not rewrite whole modules or invent unspecified requirements.

After a change, propose a normal test, a boundary test and a relevant missing/malformed-data test. Explain why each test matters. Ask me to trace the code in my own words. Only report tests as passed when there is actual execution evidence. End with the changed function, requirement covered, evidence, open questions and next small step. Help me document permitted AI assistance honestly.

Daily continuation note

Day/date: __ | Current task/function: __ | What works: __ | Evidence/ledger rows: __ | Blocker: __ | Next smallest action: __ | Reviewer: __ | Actual time today: __. Keep this note with the project so a new agent session can continue without guessing.

11. Source map and remaining inputs

Source inspected	Used for
DIT014 HT26 Assignment 2 – HarborFlow Port Intelligence Console.pdf, pp.3–11 START-HERE.txt; DATA-DICTIONARY.md	Task contracts, architecture, CodeGrade submission, minimum tests and handover. Directory structure, required/optional fields, dataset write exception and intentional errors.
main.py; data_access.py; operations.py; text_tools.py	Starter module boundaries; files are scaffolding, not completed implementations.
readme.txt; test-ledger.csv; exports README upload	Documentation/ledger structure and generated-file handling.
port_calls.csv; weather.csv	Actual date/link/numeric test conditions in the provided CSVs.
Syllabus FOP.pdf, pp.1–2	Lecture/lab schedule, group/individual learning expectations and responsible AI guidance.

Needed to finalize logistics: the Canvas deadline and complete starter ZIP. Needed for exact study exercise mapping: the relevant Lecture 11–14 slides and Lab 11–14 briefs, plus any updated schedule. Until supplied, the study map is verified at the syllabus topic/date level, not at individual exercise level.

The later lab does not disappear from the study route: complete Lab 14 on/after 23 October as consolidation and exam preparation, while the product must already meet Task 6/9 requirements at the proposed handover.